Learning Objectives

Learners will be able to...

- **Define the properties and characteristics of the set abstract data type (ADT)**
- **Create a set and explore the basic set operations**
- **Identify when to use a set**

Introduction to Set ADT

Set ADT

A set is a collection of distinct objects, called elements or members. The key feature of a set is that it does not allow duplicate elements, meaning each element is unique within the set. The set ADT is an abstraction of the mathematical concept of a finite set. It provides a way to store and manipulate a collection of distinct elements, regardless of their order.

Characteristics of Set ADT

- **Uniqueness**: Each element in a set is unique. No two elements are identical.
- **Unordered**: The elements in a set do not have a specific order. They can be stored and accessed in any order.
- **Dynamic**: Sets are dynamic in nature, meaning elements can be added or removed from the set at any time.

Basic Set Operations

- **Addition (Insertion)**: Adding a new element to the set.
- **Deletion (Removal)**: Removing an existing element from the set.
- **Membership Test**: Checking whether a particular element is present in the set.
- **Size**: Determining the number of elements in the set.
- **Iteration**: Traversing through the elements of the set.

Implementation in C++

In C++, the set ADT can be implemented using built-in classes from the Standard Library, such as `std::set`, `std::unordered_set`, and `std::multiset`. Each of these classes has its own characteristics and performance implications:

- **std::unordered_set**: Provides constant-time performance for basic operations like add, remove, and contains. It does not maintain the order of elements.
- **std::set**: Implements a set using a tree structure, providing logarithmic time performance for operations. It maintains the elements in sorted order.
- **std::multiset**: Similar to `std::set`, but allows multiple elements with the same value.

Working with Sets

Set Interface

In C++, sets are commonly implemented with the `std::set` container. We can implement a set using the Standard Template Library (STL). For now, we are going to use the `std::set` class. The `std::unordered_set` class will be covered in a later assignment. We'll start with some basic examples to demonstrate the key characteristics of sets, particularly their uniqueness and ordered nature.

Declaring a Set

There are several ways to declare a set in C++. Start with the `std::set` container and specify the data type of the elements. Then instantiate a new `std::set` object. You can pass a collection (like an array or another set) to the constructor. The values in the collection will be added as elements to the set. Add the `main` method to the code editor.

```
int main() {
    // instantiate a set with no initial elements
    std::set<int> setA;

    // instantiate a set with the elements 1, 2, and 3
    std::set<int> setB = {1, 2, 3};

    // another way to instantiate a set using an array
    int arr[] = {1, 2, 3};
    std::set<int> setC(arr, arr + 3);

    return 0;
}
```

Running the code above will not produce any output. You should not see any error messages, however.

Adding Elements

You can use the `insert` method to add elements to a set. Add the `main` method to the code editor. Remember, each element in a set is unique. Even if we try to add duplicate elements, only one will appear. Update the `main` method to look like the code below.

```
int main() {
    std::set<int> setA;
    setA.insert(1);
    setA.insert(2);
    setA.insert(2);
    setA.insert(3);
    setA.insert(3);
    setA.insert(3);

    std::cout << "Set A: ";
    for (int element : setA) {
        std::cout << element << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

You should see the following output:

```
Set A: 1 2 3
```

Checking Membership

The `find` method allows us to check if a value is a member of a set. This method returns an iterator pointing to the found element or to the end of the set if the element is not found. As long as the iterator from the `find` method is not equal to the iterator from the `end` method, we can say that the value is in the set. Update the `main` method to look like the code below.

```
int main() {
    std::set<int> setA = {1, 2, 3};

    std::cout << "Set A: ";
    for (int element : setA) {
        std::cout << element << " ";
    }
    std::cout << std::endl;

    std::cout << "Set A contains 2: " << (setA.find(2) !=
        setA.end()) << std::endl;
    std::cout << "Set A contains 4: " << (setA.find(4) !=
        setA.end()) << std::endl;

    return 0;
}
```

In this example, `1` represents true and `0` represents false. You should see the following output:

```
Set A: 1 2 3
Set A contains 2: 1
Set A contains 4: 0
```

challenge

Try this variation

Use `boolalpha` to print either `true` or `false` when checking set membership.

```cpp
int main() {
    std::set<int> setA = {1, 2, 3};

    std::cout << "Set A: ";
    for (int element : setA) {
        std::cout << element << " ";
    }
    std::cout << std::endl;

    std::cout << std::boolalpha;
    std::cout << "Set A contains 2: " << (setA.find(2) !=
        setA.end()) << std::endl;
    std::cout << "Set A contains 4: " << (setA.find(4) !=
        setA.end()) << std::endl;

    return 0;
}
```

Removing Elements from a Set

Just as you can use `insert` to add an element to a set, you can use `erase` to remove an element. In addition, the `size` method returns the size of the set. Update the `main` method to look like the code below.

```
int main() {
    std::set<int> setA = {1, 2, 3};

    std::cout << "Set A: ";
    for (int element : setA) {
        std::cout << element << " ";
    }
    std::cout << std::endl;

    std::cout << "Size before removing 2: " << setA.size() <<
        std::endl;
    setA.erase(2);
    std::cout << "Size after removing 2: " << setA.size() <<
        std::endl;

    return 0;
}
```

You should see the following output:

```
Set A: 1 2 3
Size before removing 2: 3
Size after removing 2: 2
```

Iterating Over a Set

Sets are also iterable in C++. With a range-based `for` loop, we can traverse a set. Update the `main` method to look like the code below.

```
int main() {
    std::set<int> setA = {1, 2, 3};

    std::cout << "Iterating over Set A:" << std::endl;

    for (int element : setA) {
        std::cout << element << std::endl;
    }

    return 0;
}
```

You should see the following output:

```
Iterating over Set A:
1
2
3
```

▼ Did you notice?

> Sets in C++ do maintain the order of elements, unlike in Java. The elements are stored in ascending order, and when you iterate over a set, the elements will always appear in ascending order.

The `std::set` container also provides an iterator method. This method returns an iterator to the first element, allowing us to traverse the set in a different manner. Update the main method to look like the code below.

```
int main() {
    std::set<int> setA = {1, 2, 3};

    std::cout << "Iterating over Set A:" << std::endl;

    for (auto it = setA.begin(); it != setA.end(); ++it) {
        std::cout << *it << std::endl;
    }

    return 0;
}
```

You should see the following output:

```
Iterating over Set A:
1
2
3
```

The examples for more `std::set` methods continue on the next page.

More Set Operations

Additional Set Methods

On this page, we will explore more methods provided by C++'s `std::set`.

Clearing a Set

You can erase all elements in a set using the `clear` method. Update the `main` method to look like the code below.

```
int main() {
    std::set<int> hs = {1, 2, 3};

    hs.clear();

    std::cout << "HashedSet: {";
    for (auto elem : hs) std::cout << elem << " ";
    std::cout << "}\n";

    return 0;
}
```

The output should be:

```
HashedSet: {}
```

Copying a Set

If we want to make a copy of a set, we can initialize a new set using the original set. This creates a shallow copy, and if the sets contain mutable objects, they will share references to these objects. Update the `main` method to look like the code below.

```
int main() {
    std::set<int> originalSet = {1, 2, 3};
    std::set<int> copiedSet = originalSet;

    std::cout << "Original set: ";
    for (auto elem : originalSet) std::cout << elem << " ";
    std::cout << "\nCopied set: ";
    for (auto elem : copiedSet) std::cout << elem << " ";
    std::cout << std::endl;

    return 0;
}
```

The output should be:

```
Original set: 1 2 3
Copied set: 1 2 3
```

Determining if a Set is Empty

The `empty` method returns `true` if there are no elements in a set and `false` if there are elements in a set. Update the `main` method to look like the code below.

```
int main() {
    std::set<int> setA = {1, 2, 3};
    std::set<int> setB;

    std::cout << "Set A empty: " << std::boolalpha <<
        setA.empty() << std::endl;
    std::cout << "Set B empty: " << std::boolalpha <<
        setB.empty() << std::endl;

    return 0;
}
```

The output should be:

```
Set A empty: false
Set B empty: true
```

Converting Set to Array

In C++, you can convert a `std::set` to an array by copying its elements. Create an array of the same size as the set. Copy the set into the array with the `std::copy` method. Then create a loop that runs three times, which is the size of the set and array. At each iteration compare the values in the set and array. Update the `main` method to look like the code below. **Note**, this only works when the set and the array have values sorted in ascending order.

```
int main() {
    std::set<int> hs = {1, 2, 3};
    int arr[3];
    std::copy(hs.begin(), hs.end(), arr);

    std::cout << "Conversion Verification:\n";
    bool conversionSuccess = true;
    auto setIter = hs.begin();

    for (int i = 0; i < 3; i++) {
        bool elementMatch = (arr[i] == *setIter);
        std::cout << "Position " << i << ": Set(" << *setIter
                  << ") == Array(" << arr[i] << "): "
                  << std::boolalpha << elementMatch << "\n";

        ++setIter;
    }

    return 0;
}
```

You should see the following output:

```
Conversion Verification:
Position 0: Set(1) == Array(1): true
Position 1: Set(2) == Array(2): true
Position 2: Set(3) == Array(3): true
```

Set Equality

You can use the `==` operator to determine if two sets have the same size and elements. Update the `main` method to look like the code below.

```
int main() {
    std::set<int> setA = {1, 2, 3};
    std::set<int> setB = {4, 5, 6};
    std::set<int> setC = {1, 2, 3};

    std::cout << "Are sets A and B equal? " << std::boolalpha <<
        (setA == setB) << std::endl;
    std::cout << "Are sets A and C equal? " << std::boolalpha <<
        (setA == setC) << std::endl;

    return 0;
}
```

The output should be:

```
Are sets A and B equal? false
Are sets A and C equal? true
```

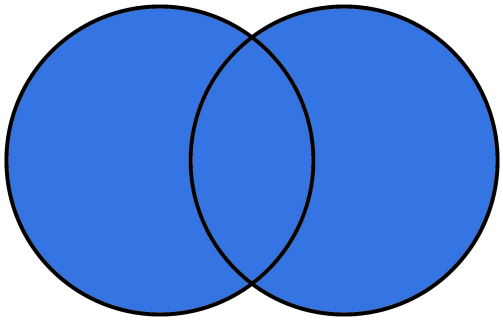
Mathematical Set Operations

Sets and Math

As previously stated, the set ADT is based on the mathematical concept of a set. That means you should be able to perform common set mathematical operations like union, intersection, difference, and subset. However, implementing a set in C++ means using methods that differ in name.

Union

The union of `setA` and `setB` is a set that contains all member elements of both `setA` and `setB`.



In C++, there is no method for the `std::set` class named “union”. Instead, we use the `insert` method to perform the union operation. The `insert` method takes all elements of a set and adds them to another set. Start by creating the set `unionSet`, which is a copy of `setA`. Call `insert` with `setB` as the argument. Then print the set `unionSet`. Add the following `main` method to the code editor.

```
int main() {
    std::set<int> setA = {1, 2, 3};
    std::set<int> setB = {3, 4, 5};

    std::set<int> unionSet(setA);
    unionSet.insert(setB.begin(), setB.end());

    std::cout << "Union of Set A and Set B: ";
    for (int num : unionSet) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

You should see the following output:

```
Union of Set A and Set B: 1 2 3 4 5
```

You can find the union of more than two sets as well. In this example, we are going to find the union of `setA`, `setB`, and `setC`. Start by creating the set `unionSet`, which is a copy of `setA`. Add all the elements from `setB` and `setC` to `unionSet`. Update the `main` method to look like the code below.

```
    int main() {
        std::set<int> setA = {1, 2};
        std::set<int> setB = {2, 3};
        std::set<int> setC = {3, 4};

        std::set<int> unionSet(setA);
        unionSet.insert(setB.begin(), setB.end());
        unionSet.insert(setC.begin(), setC.end());

        std::cout << "Union of Set A, Set B, and Set C: ";
        for (int num : unionSet) {
            std::cout << num << " ";
        }
        std::cout << std::endl;

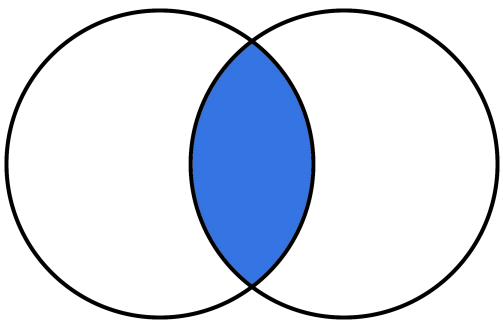
        return 0;
    }
```

You should see the following output:

```
Union of Set A, Set B, and Set C: 1 2 3 4
```

Intersection

The intersection of `setA` and `setB` is a set that contains only the elements that are common to both sets.



In C++, we can use the `std::set_intersection` algorithm to perform the intersection operation. This retains all the elements that are members of both sets. Create the intersection set by using the `set_intersection` method.

Update the `main` method to look like the code below.

```
int main() {
    std::set<int> setA = {1, 2, 3};
    std::set<int> setB = {3, 4, 5};

    std::set<int> intersection;
    std::set_intersection(setA.begin(), setA.end(),
    setB.begin(), setB.end(), std::inserter(intersection,
    intersection.begin()));

    std::cout << "Intersection of Set A and Set B: ";
    for (int num : intersection) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

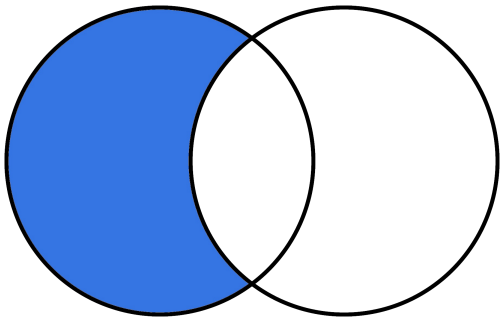
    return 0;
}
```

You should see the following output:

```
Intersection of Set A and Set B: 3
```

Difference

The difference between `setA` and `setB` is a set that contains the elements that are in `setA` but not in `setB`.



In C++, we can use the `std::set_difference` algorithm to perform the difference operation. This removes all instances of elements from one set that are in the other. Update the main method to look like the code below.

```
int main() {
    std::set<int> setA = {1, 2, 3};
    std::set<int> setB = {3, 4, 5};

    std::set<int> difference;
    std::set_difference(setA.begin(), setA.end(),
    setB.begin(), setB.end(), std::inserter(difference,
    difference.begin()));

    std::cout << "Difference of Set A and Set B: ";
    for (int num : difference) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

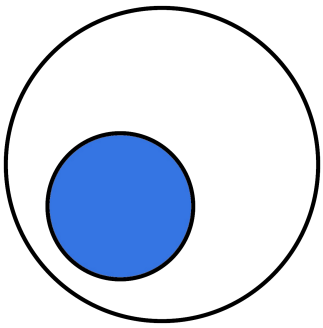
    return 0;
}
```

You should see the following output:

```
Difference of Set A and Set B: 1 2
```

Subset and Superset

`setA` is considered a subset of `setB` if all elements of `setA` are also elements of `setB`.



In C++, we can use the `std::includes` algorithm to check if one set is a subset of another. This algorithm returns a boolean value. Start by creating `setC` that has the elements `4` and `5`. Create `isSubset1`, which determines if `setA` is a subset of `setB`. Then create `isSubset2`, which determines if `setC` is a subset of `setB`. Update the `main` method to look like the code below:

```
int main() {
    std::set<int> setA = {1, 2, 3};
    std::set<int> setB = {3, 4, 5};
    std::set<int> setC = {4, 5};

    bool isSubset1 = std::includes(setB.begin(), setB.end(),
    setA.begin(), setA.end());
    bool isSubset2 = std::includes(setB.begin(), setB.end(),
    setC.begin(), setC.end());

    std::cout << "Is Set A a subset of Set B? " <<
    (isSubset1 ? "true" : "false") << std::endl;
    std::cout << "Is Set C a subset of Set B? " <<
    (isSubset2 ? "true" : "false") << std::endl;

    return 0;
}
```

You should see the following output:

```
Is Set A a subset of Set B? false
Is Set C a subset of Set B? true
```

If `setA` is a subset of `setB`, we can also say that `setB` is the **superset** of `setA`. We do not need to use a different method to determine this.

While these operations are mathematical in nature, they are also useful for solving various problems involving collections of unique elements.

Set Notation

Understanding Symbols Used with Sets

Here is a table summarizing some of the popular mathematical notation used when working with sets. While this notation is not always used in Computer Science, it is a good idea to be familiar with it.

Operation	Description	Notation	Example
Union	Combines all elements from two sets	A ∪ B	{1, 2} ∪ {2, 3} = {1, 2, 3}
Intersection	Retains only elements present in both sets	A ∩ B	{1, 2} ∩ {2, 3} = {2}
Difference	Removes elements in the second set from the first set	A - B	{1, 2} - {2, 3} = {1}
Symmetric Difference	Elements in either set, but not in both	A Δ B	{1, 2} Δ {2, 3} = {1, 3}
Subset	Checks if all elements of one set are in another set	A ⊆ B	{1, 2} ⊆ {1, 2, 3} = true
Proper Subset	Subset where the sets are not equal	A ⊂ B	{1, 2} ⊂ {1, 2, 3} = true
Superset	Checks if one set contains all elements of another set	A ⊇ B	{1, 2, 3} ⊇ {1, 2} = true
Proper Superset	Superset where the sets are not equal	A ⊃ B	{1, 2, 3} ⊃ {1, 2} = true
Disjoint	Checks if two sets have no elements in common	A ∩ B = ∅	{1, 2} ∩ {3, 4} = ∅ = true
Cardinality	The number of elements in a set	|A|	|{1, 2, 3}| = 3

C++ Cheat Sheet

Here is a cheat sheet that allows you to convert the mathematical notation for sets into the appropriate C++ method.

Operation	Description	Example C++ Code
	Combines all elements	`std::set_union(setA.begin(), setA.end(), setB.begin(), setB.end(),`

Union	from two sets	`std::inserter(resultSet, resultSet.begin()));`
Intersection	Retains only elements present in both sets	`std::set_intersection(setA.begin(), setA.end(), setB.begin(), setB.end(), std::inserter(resultSet, resultSet.begin()));`
Difference	Removes elements in the second set from the first set	`std::set_difference(setA.begin(), setA.end(), setB.begin(), setB.end(), std::inserter(resultSet, resultSet.begin()));`
Symmetric Difference	Elements in either set, but not in both	`std::set_symmetric_difference(setA.begin() setA.end(), setB.begin(), setB.end(), std::inserter(resultSet, resultSet.begin()));`
Subset	Checks if all elements of one set are in another set	`std::includes(setB.begin(), setB.end(), setA.begin(), setA.end());`
Superset	Checks if one set contains all elements of another set	`std::includes(setA.begin(), setA.end(), setB.begin(), setB.end());`

Set Usage

Applications and Use Cases of Sets

Sets are a fundamental data structure in computer science and have a wide range of applications in various fields. Understanding when and why to use sets can help you choose the right data structure for your needs and solve problems more efficiently.

When to Use Sets

- **Uniqueness**: Use sets when you need to ensure that a collection contains no duplicate elements.
- **Membership Testing**: Sets are ideal for scenarios where you frequently need to check if an element exists in a collection, as they provide efficient membership testing.
- **Mathematical Operations**: Sets are useful for performing mathematical set operations such as union, intersection, and difference.

Why Use Sets

- **Performance**: Sets, especially `std::unordered_set`, offer average constant-time performance for basic operations like add, remove, and contains, making them efficient for large datasets.
- **Simplicity**: Sets provide a simple and intuitive interface for working with unique collections of elements.
- **Flexibility**: C++ offers different implementations of the `std::set` container, such as `std::unordered_set`, `std::set`, and `std::multiset`, each with its own advantages, allowing you to choose the most suitable one for your application.

Real-Life Applications of Sets

- **Database Queries**: Sets are used in database management systems to handle unique records and perform set operations on data.
- **Search Engines**: Sets can be used to store unique keywords and perform efficient searches.
- **E-commerce**: In online shopping platforms, sets can be used to manage unique product catalogs and customer wish lists.
- **Social Networks**: Sets are used to represent connections, friends, or followers, ensuring that each connection is unique.
- **Cryptography**: Sets play a role in cryptographic algorithms and protocols, where uniqueness and set operations are crucial.

In summary, sets are a powerful and versatile data structure that are widely used in both theoretical and practical applications. They provide an efficient way to manage unique collections of elements and perform various operations, making them an essential tool in a programmer's toolkit.

Formative Assessment 1

Formative Assessment 2